ZPDPatch: Execution-Guided Program Repair from Student Submission Trajectories

ANONYMOUS AUTHOR(S)

Automated program repair for programming assignments typically treats a student's latest submission as an isolated buggy program. This snapshot omits the revisions, repeated failures, and partial improvements that led to the current code. We present ZPDPatch, a repair framework that mines supervision from authentic submission trajectories and uses execution feedback to escalate across three repair policies. The Progress adapter learns test-case-level monotonic improvements, Strict learns improvements in online-judge verdict severity, and Answer maps failed submissions to a final accepted program. At inference time, ZPDPatch invokes the adapters in that order, stops when a candidate passes all tests, and otherwise selects the candidate with the highest pass rate and smallest AST edit distance. We evaluate Qwen2.5-Coder-7B-based adapters on 997 held-out trajectories from 492 seen Project CodeNet problems and 250 trajectories from 54 problem-held-out problems. On seen problems, ZPDPatch improves repair rate from 30.7% for a trajectory-prompted zero-shot baseline to 56.6%, while producing smaller successful patches; a same-problem retrieval baseline reaches 80.2% but makes substantially larger changes. On unseen problems, ZPDPatch reaches a 72.0% repair rate versus 62.0% for zero-shot. Sequential escalation outperforms every individual adapter, but a controlled ablation finds no consistent advantage from providing full submission history rather than only the current code. The results support trajectory-mined supervision and execution-guided escalation, but do not establish a causal benefit from history conditioning itself.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation.*

Additional Key Words and Phrases: automated program repair, programming education, large language models, submission trajectories, execution-guided repair

ACM Reference Format:
Anonymous Author(s). 2027. ZPDPatch: Execution-Guided Program Repair from Student Submission Trajectories. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE '27).* ACM, New York, NY, USA, 13 pages.

1 Introduction

Programming students iteratively write, submit, execute, and revise code. Online judges make this loop scalable, but their feedback is usually limited to coarse verdicts such as `Wrong Answer`, `Runtime Error`, or `Time Limit Exceeded`. A verdict reports failure without explaining whether the latest revision made progress, repeated an earlier defect, or moved away from a viable solution. Instructors cannot inspect every trajectory and author an individualized repair at the scale of modern programming courses.

Automated program repair (APR) offers executable, code-specific feedback. Systems for programming assignments cluster peer solutions, align a student's code to a reference, or synthesize a fixed program [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. More recent systems use language models, execution diagnostics, retrieved examples, and iterative conversations [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Most of these approaches condition on the latest buggy program, possibly augmented with tests or peer programs. The earlier submissions produced by the same student are rarely treated as first-class repair context.

The motivation for using history is straightforward but unverified. Two students may arrive at similar current programs through different revisions. Their histories expose repeated errors, attempted edits, and states that they demonstrably reached. In educational terms, such reachable improvements resemble the Zone of Proximal Development (ZPD): states attainable with appropriate support but not yet reached independently [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo

2002]. We therefore ask whether repair can exploit a student's observed trajectory to produce a useful next program rather than immediately replacing it with an arbitrary correct solution. We use ZPD as a design motivation, not as a measured psychological construct or a claim about learning outcomes.

We introduce ZPDPatch, illustrated in Figure 1. It automatically converts authentic submission trajectories into three supervised repair datasets. Progress retains verdict improvements and same-verdict transitions whose per-test outcomes improve monotonically. Strict retains only transitions that improve the online-judge verdict. Answer pairs every failed submission with the trajectory's final accepted program. Three QLoRA adapters are trained independently over the same code language model and prompt. At inference time, ZPDPatch calls the adapters from narrower to broader supervision, executes each candidate, stops on acceptance, and otherwise selects a non-regressive candidate using test pass rate and AST tree edit distance (TED).

Our evaluation deliberately separates three questions that are easy to conflate: whether the complete system outperforms a zero-shot model, whether the adapters exhibit distinct behavior, and whether historical submissions themselves cause an improvement. The complete system substantially improves over zero-shot on both seen and problem-held-out data, and sequential escalation outperforms each adapter alone. However, full-history adapters do not consistently beat adapters retrained on the same examples with only the current code. This negative result narrows the paper's claim: the evidence supports supervision mined from trajectories and execution-guided composition, but not a causal advantage from placing the full trajectory in the prompt.

This paper makes the following contributions:

- We formulate trajectory-mined program repair and define three fully automatic supervision rules that capture monotonic test progress, verdict improvement, and arrival at an accepted solution.
- We design an execution-guided sequential repair procedure that combines independently trained adapters, supports early stopping, and abstains when no candidate improves over the current program.
- We conduct a problem-familiarity-aware evaluation against zero-shot and retrieval-based repair, including current-code, adapter, and sequential ablations. We report both positive system results and the absence of a consistent full-history benefit.

2 Background and Related Work

2.1 Repair for Programming Assignments

Early educational APR systems exploit the redundancy of student solutions. CLARA clusters correct programs and searches for transformations from a buggy program to a nearby cluster representative [Gulwani et al. 2018]. SARFGEN aligns student and reference programs before generating repairs [Wang et al. 2018], while Refactory uses refactoring-aware structural alignment [Hu et al. 2020]. Subsequent work extends repair and feedback generation to multiple functions, assignment-aware neural models, and diversified feedback [Choi et al. 2021; Choi and Lee 2025; Heo et al. 2023; Yi et al. 2017].

LLM-based systems reduce dependence on hand-engineered transformations. RING studies multilingual repair as code generation [Joshi et al. 2023]; PyDex and related work use language models and retrieved demonstrations for student programs [Phung et al. 2023; Zhang et al. 2022, 2024]. CREF performs conversational repair [Yang et al. 2024], FastFixer fine-tunes a repair model [Liu et al. 2024], and peer-aided repair retrieves other students' programs [Zhao et al. 2024]. Test diagnostics and counterexamples can further constrain generation [Orvalho et al. 2025; Ye et al. 2022]. LSGen combines repair-pair retrieval with iterative edit-driven generation [Dai et al. 2026]. These methods

condition on a current program, references, or diagnostics. ZPDPatch instead mines multiple improvement targets from the target student's trajectory, then evaluates whether retaining the trajectory at inference time adds value.

2.2 Adaptive Feedback and Submission Histories

Automated feedback for programming exercises spans correctness reports, hints, explanations, and repairs [Keuning et al. 2018; Narciss 2008]. Intelligent tutoring research has long examined scaffolding, help seeking, and the challenge of keeping support within a productive range [Aleven and Koedinger 2000; Chounta et al. 2017; Murray and Arroyo 2002]. Next-step hints and feedback ladders similarly offer graduated assistance rather than only final answers [Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024].

Submission histories are also used for student modeling, including knowledge tracing and context-aware variants [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015]. Our goal differs: we neither estimate latent student ability nor predict future correctness. We use observed code states as automatic evidence of reachable changes. This operationalization avoids manual student-level labels, but it does not by itself validate that a generated patch is pedagogically optimal. Section 8 makes this boundary explicit.

3 Problem Formulation

For problem p, let a student's i-th submission be $s_i = (c_i, v_i)$, where c_i is the complete source code and v_i is the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \ldots, s_n]$ is chronologically ordered and terminates at the first accepted submission. Let d_p contain the problem statement and resource constraints, and let $\mathcal{E}_p$ be the available test suite. Re-executing c_i gives a per-test outcome vector $o_i = (o_i(e))_{e \in \mathcal{E}_p}$.

A repair example consists of problem context d_p, an observed sequence h, and a complete target program y. Adapter-specific rules decide which submissions remain in h and which later submission supplies y; the two need not be adjacent in the original trajectory. Given $x = (d_p, h)$, a model generates one complete program $\hat{c}$.

We call a later state *reachable* when it occurs in the same student's observed trajectory and satisfies an adapter-specific improvement rule. Reachability is an empirical property of the data, not a guarantee that the transition is minimal, comprehensible, or caused by instructional support. Our design hypothesis is that learning from such transitions can produce repairs that preserve more of the current solution than direct answer generation.

4 Approach

4.1 Trajectory Construction

We group submissions by problem and student, sort them by timestamp, and terminate at the first `Accepted` submission. We normalize indentation, remove comments and blank lines, and remove consecutive submissions that are identical after normalization. Until adapter-specific filtering, we preserve complete source code, original verdict, order, and all observed history. We impose no maximum history length or transition count. We also re-execute every submission on the available tests and cache per-test outcomes for deterministic reuse in dataset construction and evaluation.

4.2 Adapter-Specific Supervision

We order verdict severity as $\mathrm{sev}(\texttt{AC}) = 0$, $\mathrm{sev}(\texttt{WA}) = 1$, $\mathrm{sev}(\texttt{TLE/MLE}) = 2$, $\mathrm{sev}(\texttt{RE}) = 3$, $\mathrm{sev}(\texttt{CE}) = 4$, and 5 for internal failures. For equal, non-empty test keys, $o_t \preceq_{\mathrm{sev}} o_r$ iff no test outcome in o_t is

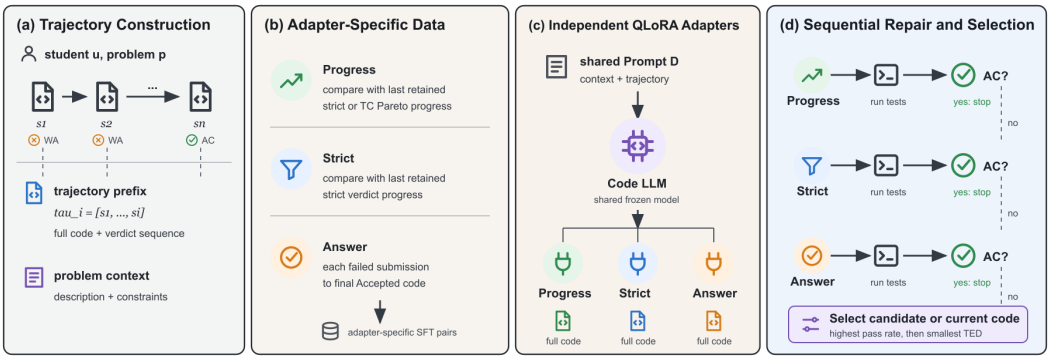

Fig. 1. Overview of ZPDPatch. Authentic submission trajectories are normalized and re-executed. Adapter-specific rules construct supervision for Progress, Strict, and Answer. Independently trained adapters generate candidates sequentially; execution outcomes determine early stopping and final selection.

more severe than its counterpart in o_r. Test-case progress is the Pareto condition

$$\mathrm{TCProg}(r,t) = \mathbf{1}[o_t \preceq_{\mathrm{sev}} o_r \wedge o_t \neq o_r]\,. \tag{1}$$

Answer. If s_n is accepted, every preceding non-accepted submission is independently paired with c_n. For $\tau = [S_1, \ldots, S_9]$, the rule produces $([S_1], c_9), \ldots, ([S_8], c_9)$. Thus, Answer learns a direct failed-program-to-answer mapping and receives no multi-submission history.

Strict. Starting with $R = [s_1]$, we scan the trajectory and append s_t only when its verdict is strictly better than the last retained submission $s_r = R[-1]$: $\mathrm{sev}(v_t) < \mathrm{sev}(v_r)$. If $R = [r_1, \ldots, r_m]$, we create $([r_1, \ldots, r_{k-1}], c_{r_k})$ for every $k = 2, \ldots, m$. The comparison is against the last retained state, not necessarily the immediately preceding original submission.

Progress. Progress uses the same retained scan but appends s_t when either its verdict strictly improves or $v_t = v_r$ and $\mathrm{TCProg}(r,t) = 1$. It therefore captures monotonic per-test progress that is hidden by an unchanged coarse verdict. Prefix-target construction is otherwise identical to Strict.

The datasets are constructed independently and may overlap. A strict transition is also a progress transition, and final accepted transitions can occur in both. The objectives nevertheless differ: Answer learns direct completion from one failed program, whereas Strict and Progress learn transitions from retained prefixes.

4.3 Prompt and Adapter Training

All adapters use the same prompt. The system message contains the problem statement and resource limits. Each retained submission is represented by its complete code and online verdict; cached per-test outcomes and an AST/line edit summary are included when available. The final user message is the current program and the assistant target is one complete Python program. The prompt never names an adapter role, so specialization can arise only from supervision.

For adapter $a \in \{\mathrm{prog}, \mathrm{strict}, \mathrm{ans}\}$, let $\mathcal{D}_a$ be its dataset, x_i the prompt, and y_i the target tokens. We minimize the target-only supervised fine-tuning objective

$$\mathcal{L}_a = -\frac{1}{|\mathcal{D}_a|} \sum_{(x_i, y_i) \in \mathcal{D}_a} \frac{1}{|y_i|} \sum_{k=1}^{|y_i|} \log P_{\theta,\phi_a}(y_{i,k} \mid y_{i,<k}, x_i), \tag{2}$$

where θ is the frozen base model and ϕ_a is an adapter's QLoRA parameter set. Prompt tokens are masked from the loss. The adapters are trained and stored independently.

4.4 Execution-Guided Sequential Repair

At inference time, we execute the observed submissions to populate consistent feedback and then invoke Progress, Strict, and Answer in that order. Each adapter generates one complete program. A candidate that passes every test is returned immediately. Otherwise, its verdict, pass rate, per-test outcomes, and improvement status are appended to the next-stage prompt. For a no-op or regression, only the execution result is passed forward, avoiding reinforcement of the candidate code.

If all candidates fail, the selector includes the current program c_i as a fallback:

$$C = \{c_i, c^{\text{prog}}, c^{\text{strict}}, c^{\text{ans}}\}. \tag{3}$$

Let $\text{Pass}(c)$ be the fraction of tests passed and $\text{ted}(c_i, c)$ the Python AST tree edit distance. Selection is lexicographic:

$$c^* = \arg\max_{c \in C} \left(\text{Pass}(c), -\text{ted}(c_i, c)\right). \tag{4}$$

The selector first maximizes executed correctness and then minimizes change. If no candidate improves on c_i, the unchanged fallback has TED zero and the system abstains.

5 Experimental Design

5.1 Research Questions

RQ1: Repair Effectiveness. How effectively does ZPDPatch repair held-out trajectories from problems observed during training, compared with zero-shot and retrieval-based repair?

RQ2: Trajectory Conditioning. Does access to the full submission prefix improve repair over training and inference with only the current code?

RQ3: Adapter Specialization. Do Progress, Strict, and Answer exhibit different repair coverage and patch size?

RQ4: Sequential Escalation. Does execution-guided composition improve effectiveness without always invoking all adapters?

RQ5: Problem Generalization. Does ZPDPatch retain an advantage on problems completely excluded from training?

5.2 Dataset Construction and Splits

We combine Python submissions, statements, metadata, and test cases for problems in Project CodeNet's Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the Python800 reference set. We remove post-acceptance submissions and consecutive normalized duplicates. Users must appear in at least three problems and each retained problem must contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests.

Before splitting, we tokenize every possible Answer, Strict, conservative Progress, and final-evaluation prompt-target configuration. If any configuration exceeds 4,096 tokens, we exclude the entire trajectory rather than truncating code or history. This removes 2,896 trajectories and leaves 18,558. No later `max_history`, transition-count, code, or test truncation is applied.

We sort problems by eligible trajectory count and assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*. Within every Seen problem, we reserve at least

Table 1. Dataset statistics after whole-trajectory context filtering. "Prefix" counts possible raw prefixes before adapter-specific filtering.

Group	Role	Problems	Tests	Traj.	Sub.	Prefix	Users
Seen	Train	492	9,446	14,638	58,872	44,234	4,072
	Validation			1,830	7,283	5,453	1,283
	Test			1,830	7,068	5,238	1,291
Unseen	Test	54	642	260	965	705	237

Table 2. Adapter supervision datasets. Validation examples are excluded from training.

Adapter	Train examples	Validation examples
PROGRESS	21,416	2,685
STRICT	16,973	2,116
ANSWER	40,454	4,965

one trajectory for Train, Validation, and Test, then obtain a deterministic 80:10:10 trajectory split. All 260 Unseen trajectories remain problem-held-out test data. Table 1 reports the resulting corpus. Users may occur in more than one split; RQ5 is problem-held-out, not user-held-out.

Adapter construction yields the training and validation sizes in Table 2. Final evaluation uses the prefix ending at the last non-accepted submission and the final accepted program as the oracle. We retain only cases for which re-execution confirms the current program as non-accepted and the oracle as accepted, leaving 997 Seen and 250 Unseen repair instances.

5.3 Baselines

Zero-shot. The zero-shot baseline uses the same Qwen base model and prompt as ZPDPATCH without fine-tuning. If a candidate fails, its verdict and number of passed tests are added to the conversation before the next attempt. It receives at most three generations, matching ZPDPATCH's maximum budget. Because it receives the full trajectory, comparison with zero-shot measures the complete effect of learned adapters and sequential composition, not history in isolation.

LSGen. LSGen is an iterative retrieval-based solution generator [Dai et al. 2026]. We preserve its repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval, while replacing hard-coded infrastructure with our common dataset and runner. For a Seen query, the retrieval database contains buggy/correct pairs from other Seen-Train trajectories of the same problem; the same student and example are excluded. LSGen retrieves five pairs and performs at most three iterations. We do not evaluate it on Unseen problems because exposing same-problem correct programs would violate problem holdout.

All approaches use the same base LLM, public tests, 2.5-second per-test timeout, 2GB memory limit, and maximum of three candidate generations.

5.4 Implementation

We use Qwen2.5-Coder-7B-Instruct [Hui et al. 2024] and train each adapter for one epoch with 4-bit QLoRA [Dettmers et al. 2023]. The base weights use NF4 double quantization and BF16 computation. LoRA targets the q, k, v, o attention projections and gate/up/down MLP projections with rank 16, scaling 32, and dropout 0.05. Optimization uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine

scheduling, 0.03 warmup ratio, gradient checkpointing, micro-batches of 2, eight accumulation steps, and seed 2027. We select the lowest validation-loss checkpoint. Greedy decoding uses all model context remaining after the input. Training and generation run on one NVIDIA RTX 5090.

A single Python runner executes original and generated programs. We cache outcomes by problem, code, test set, timeout, and memory configuration and reuse the cache across supervision construction, stage feedback, and evaluation. Candidate generation can proceed in parallel, but candidate execution and recorded outcomes are shared so that an identical program cannot receive inconsistent labels under CPU contention.

5.5 Ablations

For RQ2, we construct adapter-specific Strict and Progress examples for both Seen and Unseen tests. *Full Trajectory* uses each retained prefix. *Current Code Only* retrains an adapter on identical examples and targets after removing every submission before the current one; its displayed position is reset to S_1 to prevent index leakage. Answer is excluded because its inputs already contain one submission. This design isolates the presence of history while holding targets, counts, model, and hyperparameters fixed.

For RQ3, each adapter independently generates one candidate for the same 997 Seen instances. For RQ4, we compare these individual candidates with the full Progress–Strict–Answer sequence, including stage feedback, early stopping, and fallback selection.

5.6 Metrics and Statistical Analysis

For M instances, let c_m be the current program and $\hat{c}_m$ the selected result. $\mathrm{Pass}(c) \in [0, 1]$ is the test pass fraction. Pass Rate (PR) averages $\mathrm{Pass}(\hat{c}_m)$; Repair Rate (RR) averages $\mathbf{1}[\mathrm{Pass}(\hat{c}_m) = 1]$; Improvement Rate (IR) averages $\mathbf{1}[\mathrm{Pass}(\hat{c}_m) > \mathrm{Pass}(c_m)]$.

Average Time Taken (ATT) is weighted wall-clock time per repaired input, measured from generation through execution and selection for all inputs of a problem. Shared cache construction, model loading, training, and LSGen database construction are offline preparation and excluded.

For successfully repaired, parseable programs, $\mathrm{TED}_{B\to F}$ measures AST distance from current to fixed code and $\mathrm{TED}_{F\to O}$ from fixed code to the recorded accepted oracle. RQ2 uses $\mathrm{TED}_{F\to T}$, where T is the identical adapter-specific recorded target shared by Full and Current configurations.

We report Wilson 95% intervals for RR and IR. Paired RR comparisons use two-sided exact McNemar tests. Paired PR and TED differences use 10,000 bootstrap resamples with seed 2027. We apply Holm correction within each planned family of RR comparisons: the three RQ1 methods, four RQ2 adapter-split comparisons, three RQ3 adapter comparisons, and three RQ4 comparisons against Sequential. Unless stated otherwise, percentages are absolute percentage points.

6 Results

6.1 RQ1: Repair Effectiveness

Table 3 summarizes RQ1 and RQ5. On Seen problems, ZPDPatch repairs 564 of 997 programs (56.6%), compared with 306 (30.7%) for Zero-shot. Their Wilson RR intervals are [53.5, 59.6] and [27.9, 33.6], respectively. ZPDPatch alone repairs 301 instances, whereas Zero-shot alone repairs 43 (Holm-adjusted McNemar $p = 1.64 \times 10^{-48}$). PR increases by 8.57 points (95% bootstrap CI [6.99, 10.22]) and IR by 20.9 points.

LSGen achieves the highest Seen execution correctness: 88.0% PR, 80.2% RR, and 82.4% IR. It alone repairs 306 instances, while ZPDPatch alone repairs 70 (Holm-adjusted $p = 2.70 \times 10^{-36}$). The tradeoff is patch size. ZPDPatch's mean $\mathrm{TED}_{B\to F}$ is 19.71, versus 27.25 for Zero-shot and 88.27 for LSGen. Among jointly repaired parseable instances, ZPDPatch reduces TED by 8.07 relative to

Table 3. Main repair results. PR, RR, and IR are percentages; ATT is seconds. TED is computed on successfully repaired, parseable programs.

Split	Method	PR	RR	IR	ATT	$TED_{B\to F}$	$TED_{F\to O}$
Seen	Zero-shot	76.3	30.7	43.7	12.87	27.25	27.67
	LSGen	**88.0**	**80.2**	**82.4**	13.00	88.27	83.23
	ZPDPatch	84.9	56.6	64.6	**12.16**	**19.71**	**21.89**
Unseen	Zero-shot	75.3	62.0	68.8	**3.89**	17.46	15.80
	ZPDPatch	**83.0**	**72.0**	**76.8**	6.17	**11.66**	**12.54**

Table 4. RQ2 trajectory-conditioning results. PR, RR, and IR are percentages. Current and Full use identical examples and targets.

Split	Adapter	Input	N	PR	RR	IR	$TED_{F\to T}$
Seen	Strict	Current	2,151	56.8	31.2	54.6	40.23
	Strict	Full	2,151	57.5	30.5	53.3	41.05
	Progress	Current	2,674	58.4	26.3	49.7	36.23
	Progress	Full	2,674	57.4	25.9	49.1	33.15
Unseen	Strict	Current	322	66.7	56.2	71.4	20.20
	Strict	Full	322	65.4	53.4	70.8	19.27
	Progress	Current	378	65.7	52.1	66.4	17.63
	Progress	Full	378	66.5	53.7	68.3	18.16

Zero-shot (95% CI [4.53, 12.14]) and by 43.41 relative to LSGen (95% CI [38.20, 48.94]). ZPDPatch also has the lowest Seen ATT at 12.16 seconds, though timing differences are small.

Answer to RQ1. ZPDPatch substantially outperforms a trajectory-prompted zero-shot model and produces smaller successful patches. Same-problem retrieval remains much stronger in repair coverage, so ZPDPatch is complementary to, rather than a replacement for, a correct-solution database.

6.2 RQ2: Trajectory Conditioning

Table 4 shows no consistent advantage for Full Trajectory. On Seen Strict examples, Full raises PR by 0.7 points but lowers RR and IR by 0.7 and 1.3. On Seen Progress, it lowers PR, RR, and IR by 1.0, 0.4, and 0.6. On Unseen data, Full is worse for Strict but better for Progress. Target TED is also mixed: Full is closer to the recorded target for Seen Progress and Unseen Strict, but farther in the other conditions. None of the four paired RR differences is significant after Holm correction (minimum adjusted $p = 0.313$).

Answer to RQ2. Providing full submission history does not consistently improve repair over current code alone. RQ1 therefore cannot be interpreted as evidence for a causal history-conditioning effect; it measures the complete learned and sequential system.

6.3 RQ3: Adapter Specialization

Table 5 exposes a coverage-size tradeoff. Answer has the highest PR, RR, and IR, but also the largest mean patch at TED 20.22. Progress has lower coverage but the smallest patch at TED 15.85. Strict and Progress have statistically indistinguishable RR (adjusted $p = 0.857$). Answer repairs

Table 5. RQ3 individual-adapter results on 997 Seen instances.

Adapter	PR	RR	IR	$TED_{B\to F}$	$TED_{F\to O}$
Progress	73.4	44.8	52.0	**15.85**	**15.89**
Strict	74.8	44.5	51.2	17.45	18.38
Answer	**77.5**	**50.1**	**57.5**	20.22	20.61

Table 6. RQ4 sequential escalation on 997 Seen instances. "Candidates" is the mean number generated.

Configuration	PR	RR	IR	ATT	Candidates
Progress	73.4	44.8	52.0	10.91	1.00
Strict	74.8	44.5	51.2	10.95	1.00
Answer	77.5	50.1	57.5	**9.99**	1.00
Sequential	**84.9**	**56.6**	**64.6**	12.16	2.05

5.2 points more than Progress and 5.5 points more than Strict (adjusted $p = 6.21 \times 10^{-4}$ and 1.92×10^{-4}, respectively). The ordered increase in patch size is consistent with broader targets, but Strict does not form a clearly distinct coverage regime.

Answer to RQ3. The supervision rules induce different repair-size behavior: direct answer supervision produces the broadest and most successful individual adapter, while progress supervision produces the smallest successful patches. Verdict-only Strict is not clearly separated from Progress in repair coverage.

6.4 RQ4: Sequential Escalation

Sequential repair reaches 56.6% RR, 6.5 points above the strongest individual adapter, Answer. Sequential alone repairs 119 cases and Answer alone repairs 54 (Holm-adjusted $p = 8.57 \times 10^{-7}$). Its advantage over Progress and Strict is also significant (adjusted $p = 1.64 \times 10^{-22}$ and 1.87×10^{-21}).

Early stopping limits cost. Of 997 inputs, 439 stop after Progress and another 66 after Strict. The system generates 2.05 candidates on average instead of always generating three. ATT is 12.16 seconds, only 2.17 seconds above the fastest individual adapter. The final selector returns a Progress candidate for 484 instances, Strict for 88, Answer for 72, and the unchanged fallback for 353.

Answer to RQ4. Execution-guided escalation improves both complete repair and partial improvement over every single adapter. Early stopping avoids a fixed three-generation cost, although the sequence still averages roughly twice the generation work of an individual adapter.

6.5 RQ5: Problem Generalization

On 250 Unseen instances, ZPDPatch obtains 83.0% PR, 72.0% RR, and 76.8% IR, versus 75.3%, 62.0%, and 68.8% for Zero-shot. ZPDPatch alone repairs 42 cases and Zero-shot alone repairs 17 (McNemar $p = 0.00155$). The PR difference is 7.65 points (95% bootstrap CI [3.59, 11.93]). ZPDPatch also reduces mean successful-patch TED from 17.46 to 11.66. Among joint repairs, its patch is 6.33 TED units smaller on average (95% CI [3.31, 9.77]).

Answer to RQ5. ZPDPatch's advantage over Zero-shot persists on problems completely excluded from adapter training. Because the Unseen group deliberately contains lower-resource problems,

its higher absolute RR must not be interpreted as evidence that unseen problems are easier or that familiarity harms repair.

7 Discussion

7.1 What Produces the Observed Improvement?

The ablations separate three mechanisms. First, adapter training matters at the system level: ZPDPatch strongly outperforms a zero-shot model that already receives the trajectory prompt. Second, sequential execution matters: combining candidates improves over every individual adapter. Third, full-history conditioning does *not* show a stable benefit when supervision and targets are controlled. The strongest defensible conclusion is therefore that trajectories are useful as a source of automatically mined repair transitions, while their value as inference context remains unresolved.

Several mechanisms may explain the null history result. Long histories can dilute the current defect, repeated code may consume attention without adding information, and retained prefixes encode states selected by rules rather than necessarily causal edits. The current-code adapter may also internalize common transition patterns from many examples without needing an individual history at inference. A stronger test would hold the current code exactly constant and pair it with counterfactual histories that imply different next actions.

7.2 Coverage Versus Patch Size

LSGen's 80.2% RR demonstrates the value of same-problem correct programs. Its successful patches are, however, more than four times as distant from the buggy program as ZPDPatch's by mean AST TED. This difference should not be interpreted as automatic pedagogical superiority: small patches can preserve a student's structure, but TED does not measure explanation quality or learning. The result establishes a software-change tradeoff. Retrieval maximizes coverage when a same-problem database exists; learned adapters generate more conservative repairs and remain applicable when the problem is held out.

The adapters also reveal that broader target supervision increases both coverage and modification size. Answer is the strongest individual adapter, while Progress produces the smallest successful changes. Strict adds little separation beyond Progress, suggesting that coarse verdict severity is an insufficient intermediate objective. Future variants could learn from quantitative pass-rate deltas, fault-localization signals, or edit locality rather than a hand-ordered verdict scale.

7.3 Implications for Educational Use

ZPDPatch produces complete programs, not natural-language hints. A deployment could expose a patch as a diff, localize the changed region, or translate it into a hint ladder. It should not silently overwrite student work. The fallback behavior is useful here: when no candidate improves executed correctness, the system returns the current program rather than presenting a regression as help.

No result in this paper measures student learning, comprehension, or the appropriateness of assistance. The ZPD framing motivates progressively broader repair, but validating educational benefit requires a study with learners and outcomes such as independent transfer, time to solution, and help-seeking behavior. Until then, ZPDPatch should be understood as a repair system whose outputs may support feedback authoring, not as a validated tutoring intervention.

8 Threats to Validity

Construct validity. Passing the available tests is a proxy for semantic correctness and may admit overfitting. We reduce this risk by requiring the recorded oracle to pass the same suite, but hidden tests could change absolute rates. AST TED measures structural change, not readability, conceptual

difficulty, or educational value. "Reachable" means observed later in a trajectory; it does not imply that the generated transition lies within an individual learner's ZPD. We therefore avoid claims about learning outcomes.

Internal validity. Dataset construction, verdict ordering, normalization, runner limits, and candidate selection can affect results. We use one runner and shared outcome cache across methods, preserve full trajectories after filtering, and compare paired outputs on identical instances. LSGen required infrastructure adaptation; although its retrieval and generation logic is retained, our integration may differ from its original environment. Greedy decoding and one training seed improve reproducibility but do not characterize variance across seeds or sampling strategies.

Data leakage and dependence. Unseen problems are completely excluded from training and retrieval. Seen splits share problem identities by design, and users may occur across splits; the benchmark is not user-held-out. Multiple trajectories from the same problem or user are statistically dependent, whereas our confidence intervals resample instances. The paired tests remain useful for method comparison on the benchmark, but their intervals may be narrower than problem-clustered estimates. A future evaluation should bootstrap by problem and add user-held-out splits.

External validity. The study covers Python solutions from Project CodeNet, one 7B code model, one GPU, and programming-contest-style tasks. Results may not transfer to multi-file projects, other languages, industrial defects, private autograders, or novice courses with different submission behavior. The Unseen set consists of lower-volume problems by construction and should not be treated as a random sample of all new assignments.

Conclusion validity. We define explicit metric families and apply Holm correction to RR comparisons, but RQ3 and RQ4 remain exploratory ablations. PR and TED bootstrap intervals use deterministic outputs from one trained checkpoint. Replication with multiple training seeds, models, and test suites is needed before treating the measured effect sizes as universal.

Ethical considerations. The study analyzes a publicly released code dataset and performs no intervention with learners. We do not attempt to re-identify users, and derived evaluation reports need only pseudonymous identifiers. Nevertheless, source-code histories can carry authorship signals; a public artifact should minimize identifiers and follow Project CodeNet's distribution terms.

9 Conclusion

We presented ZPDPatch, an execution-guided APR framework trained from authentic student submission trajectories. Three automatic supervision rules expose monotonic test progress, verdict improvement, and final accepted solutions. Sequential composition raises Seen repair rate from 50.1% for the strongest individual adapter to 56.6%, while early stopping limits generation to 2.05 candidates on average. The complete system outperforms a trajectory-prompted zero-shot model on Seen and problem-held-out Unseen data and produces smaller successful patches, although same-problem retrieval achieves higher coverage.

The current evidence also sets an important boundary. Full trajectory prefixes do not consistently outperform current code alone. Thus, the results support trajectory-*mined* supervision and execution-guided escalation, not a causal benefit from history conditioning. Future work should identify when history is informative, test counterfactual histories for identical current code, evaluate clustered uncertainty across problems, and study whether the resulting patches improve human learning.

10 Data Availability

The submission will be accompanied by an anonymized replication package containing the implementation, prompts, data-construction and experiment scripts, split manifests, configuration records, and aggregate/per-instance evaluation outputs needed to reproduce the reported analyses. Raw Project CodeNet submissions and tests will not be redistributed; the package will document how to obtain them from their original source and regenerate derived artifacts. Model checkpoints and large execution caches will be provided through an anonymous archival link where licensing and storage permit, or accompanied by deterministic regeneration commands and checksums. Upon acceptance, the authors intend to release the package publicly.

References

Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.

Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.

Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The "Grey Area": A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.

Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes. Cambridge, MA: Harvard University Press* (1978).

Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.

Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.

Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

Hasnain Heickal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).

Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE, 388–398.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.

Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.

Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.

Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.

Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).

Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).

Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.

Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladmir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).

Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.

Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.

Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024).